

МИНОБРНАУКИ РОССИИ
*Федеральное государственное бюджетное образовательное учреждение
высшего образования*
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт Информационных технологий (ИТ)
Кафедра Математического обеспечения и стандартизации информационных
технологий (МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 3
по дисциплине
«Технология разработки программных приложений»

Тема: «Системы сборки»

Выполнил студент группы ИКБО-50-23

Астахов С.П.

Принял

Степанов П.В.

Практическая работа выполнена

«16» 03 2025 г.

(подпись студента)

«Зачтено»

«__» _____ 2025 г.

(подпись руководителя)

Москва 2025

Содержание

1	Формулировка задачи	3
2	Решение	4
3	Контрольные вопросы.....	8
4	Вывод.....	10

1 Формулировка задачи

Задания:

Необходимо клонировать (или форкнуть) git-репозиторий согласно варианту, и выполнить следующие задания:

- 1) Найти отсутствующую зависимость и указать ее в соответствующем блоке в build.gradle, чтобы проект снова начал собираться
- 2) В некоторых классах поправить имя пакета
- 3) Собрать документацию проекта, найти в ней запросы состояния и сущности по идентификатору
- 4) Собрать jar со всеми зависимостями (так называемый UberJar), после чего запустить приложение. По умолчанию, сервер стартует на порту 8080.
- 5) Запросить состояние запущенного сервера (GET запрос по адресу <http://localhost:8080>)
- 6) Запросить сущность по идентификатору (GET запрос по адресу: <http://localhost:8080/сущность/идентификатор>). Идентификатором будут 3 последних цифры в серийном номере вашего студенческого билета.
- 7) В задаче shadowJar добавить к jar-файлу вашу фамилию
- 8) Выполнить задачу checkstyleMain. Посмотреть сгенерированный отчет. Устранить ошибки оформления кода.

Собственный вариант: 2) репозиторий: <https://github.com/rtu-mirea/trpp-second-2>, сущность, ru.mirea.entity.Student

2 Решение

```
PS C:\Users\cotei\OneDrive\Desktop\Rtu_MIREA\2 курс\4 семестр\Технология разработки программных приложений\3ПР> git clone https://github.com/rtu-mirea/trpp-second-2.git
Cloning into 'trpp-second-2'...
remote: Enumerating objects: 40, done.
remote: Counting objects: 100% (40/40), done.
remote: Compressing objects: 100% (28/28), done.
remote: Total 40 (delta 0), reused 40 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (40/40), 107.40 KiB | 886.00 KiB/s, done.
PS C:\Users\cotei\OneDrive\Desktop\Rtu_MIREA\2 курс\4 семестр\Технология разработки программных приложений\3ПР> |
```

Рисунок 1 - Клонирование репозитория

```
23 dependencies {
24     annotationProcessor 'org.projectlombok:lombok:1.18.18'
25     compileOnly 'org.projectlombok:lombok:1.18.18'
26
27     implementation("io.micronaut:micronaut-runtime")
28     implementation("io.micronaut:micronaut-validation")
29     implementation("io.micronaut:micronaut-http-client")
30     implementation("javax.annotation:javax.annotation-api")
31     implementation('org.apache.logging.log4j:log4j-core:2.17.1') // update
32     implementation('com.opencsv:opencsv:5.10') // new
33     runtimeOnly("org.apache.logging.log4j:log4j-api:2.12.1")
34     runtimeOnly("org.apache.logging.log4j:log4j-slf4j-impl:2.12.1")
35 }
36 }
```

Рисунок 2 – Добавление/обновление нужных зависимостей

```
src
├── main
│   ├── java
│   │   └── ru.mirea.trpp_second_2
│   │       ├── controllers
│   │       │   ├── HealthController
│   │       │   └── StudentController
│   │       ├── entity
│   │       │   └── Application
│   │       ├── resources
│   │       └── test
```

Рисунок 3 – Правка имен пакетов

На рисунках 4-5 покажем запрос состояния и сущности по идентификатору через документацию java - javaDoc

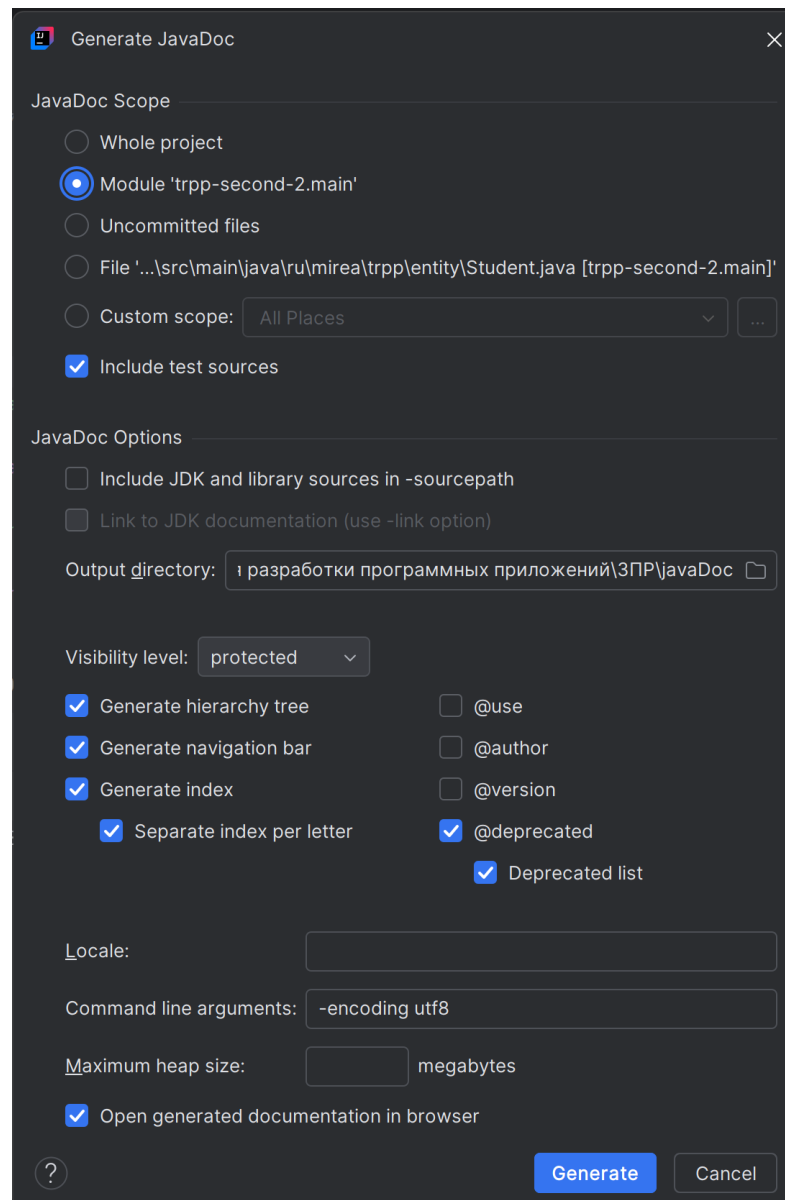


Рисунок 4 – Создание javadoc

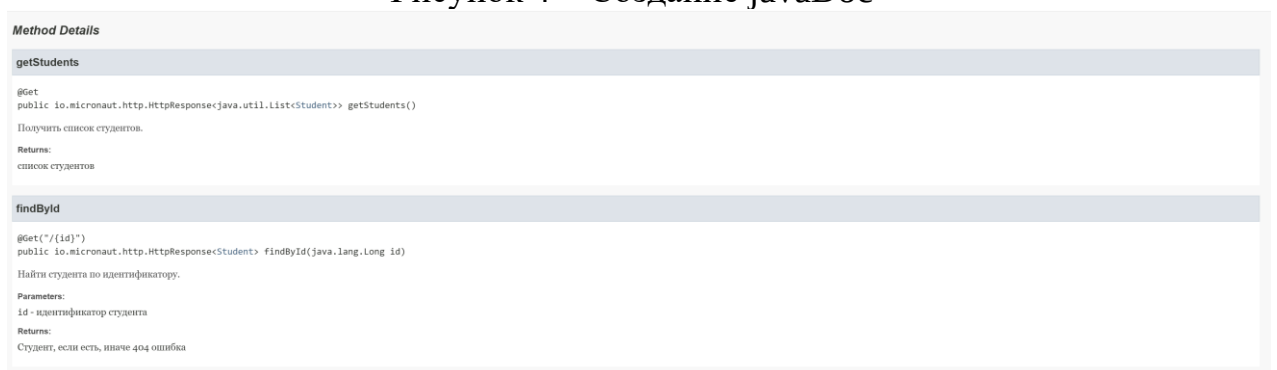


Рисунок 5 – Документация запросов состояния и сущности по идентификатору

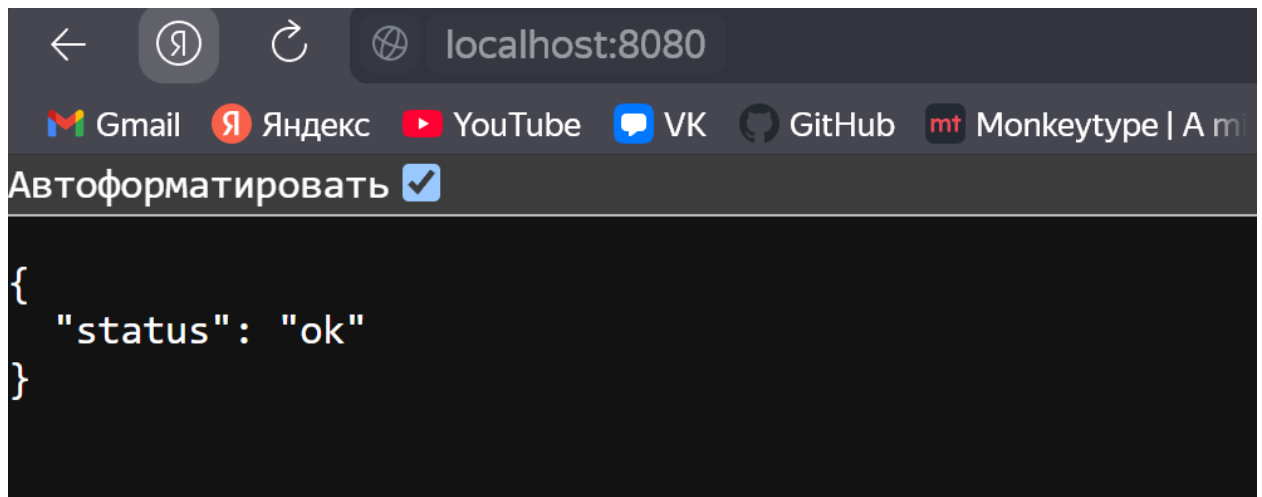


Рисунок 6 – Запуск приложения на сервер 8080

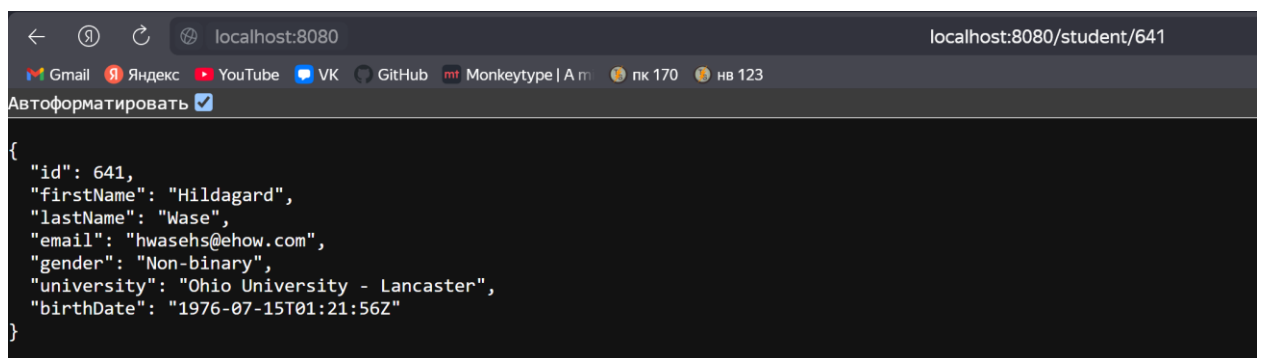


Рисунок 7 – Запрос сущности по идентификатору

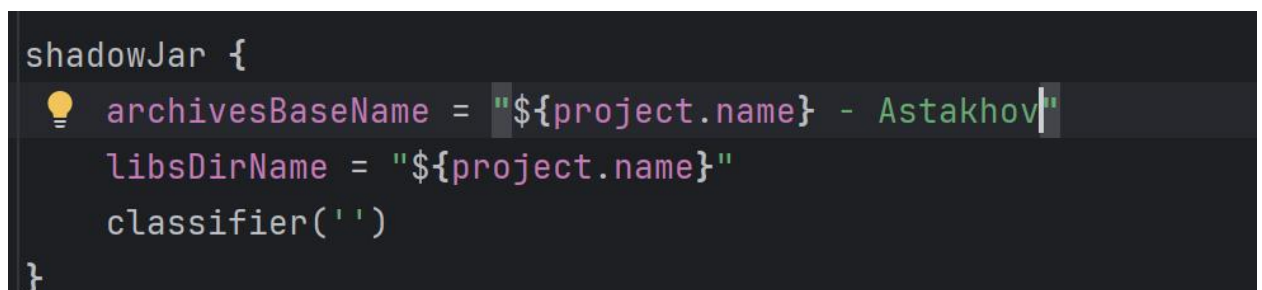


Рисунок 8 – Фамилия в shadowJar в buildGradle

```
7:00:51: Executing 'checkstyleMain'...

> Task :compileJava UP-TO-DATE
> Task :processResources UP-TO-DATE
> Task :classes UP-TO-DATE
> Task :checkstyleMain UP-TO-DATE

BUILD SUCCESSFUL in 103ms
3 actionable tasks: 3 up-to-date
7:00:52: Execution finished 'checkstyleMain'.
```

Рисунок 9 - CheckstyleMain

3 Контрольные вопросы

1. Чем компиляция отличается от сборки?

Компиляция – это процесс преобразования исходного кода в машинный код или промежуточный код (например, байт-код для Java). Компилятор выполняет синтаксический анализ генерацию и оптимизацию кода.

Сборка – это более широкий процесс, который включает в себя компиляцию, а также другие этапы, такие как линковка (связывание объектных файлов), упаковка в архив (например, Jar), выполнение тестов, копирование ресурсов и т.д. Сборка создает готовый к использованию продукт.

2. Что такое система сборки?

Система сборки – это инструмент, который автоматизирует процесс сборки программного проекта. Она управляет компиляцией, линковкой, упаковкой, тестированием и другими задачами.

3. Что такое репозиторий?

Репозиторий – это хранилище, где хранится исходный код проекта, а также его метаданные (например, история изменений, версии, зависимости). Репозитории бывают:

- Локальный репозиторий
- Удаленный репозиторий

4. Как указать зависимости проекта?

Зависимости проекта указываются в конфигурационных файлах, которые используются системой сборки. Например:

В Gradle зависимости указываются в файле `build.gradle` в блоке `Dependencies`

5. Что такое gradle?

Gradle – это система сборки с открытым исходным кодом, которая использует Groovy или Kotlin DSL для написания скриптов сборки.

6. Что такое maven?

Maven – это система сборки и управления зависимостями для Java-проектов. Она использует XML-файл(`pom.xml`) для описания конфигурации проекта, зависимостей и этапов сборки.

4 Вывод

В ходе полученной работы мы изучили систему сборки gradle. Покопались во внутренних настройках проекта в build.gradle, узнали, что такое зависимости, артефакты, плагины и исправили недочеты. Также мы реализовали запрос на состояние по идентификатору, путем get запросу.